Efficient OPC UA binary encoding considerations for embedded devices

Chris Paul Iatrou
Technical University of Dresden
Chair for Control Systems Engineering
Email: Chris_Paul.Iatrou@tu-dresden.de

Leon Urbas
Technical University of Dresden
Chair for Control Systems Engineering
Leon.Urbas@tu-dresden.de

***Abstract*—The implementation of software based OPC UA servers on computing platforms in actuators and sensors of the field layer remains a challenge due to the protocols high memory and bandwidth prerequisites. This paper discusses encoding aspects of OPC UA binary data representations for devices with limited random access memory and low fieldbus throughput, especially focusing on single-chip microcomputing platforms. Efficient and machine-friendly binary representations of OPC UA data models are derived by examining the structure of OPC UA data and optimizing their representation in regard to 8 bit serial, non-volatile memory components. Transport compression mechanisms, aiming to reduce bandwidth requirements in crowded or low bandwidth networks, are also introduced. While the memory usage of information models could be significantly reduced to (116 kB for Namespace 0), it was shown transport compression cannot yield bandwidth improvements unless data is compressed as a service.**

I. Introduction

The integration of actor and sensors into a semantically active information hierarchy is a major challenge for next generation automation networks. OPC Unified Architecture (OPC UA) was introduced by the OPC Foundation as a successor to the widespread OPC protocol to meet this challenge.[1], [2] OPC UA enables using extensive data modeling capabilities, object-orientation and inter-platform communications in the coming generations of industrial networks.[3] The protocol is tailored to fulfill the role of a semantic integration and interaction carrier between heterogeneous components in decentralized, linked data structures [4] as well as vendor independent, inter-system communications [5].

In order for OPC UA to fulfill these roles in novel applications, the usage of computationally strong systems is usually implied. Implementing OPC UA servers on microcomputing architectures of field devices like actuators and sensors remains challenging due to the lack of memory and computing resources on those platforms. Known implementations are either not standard compliant [6] or result in mayor impairments of the protocols functionality [7]. As a consequence of the protocols object-oriented design and its extensive data (meta-)modeling capabilities, arising problems are first and foremost attributable to in-memory, runtime data representation. These memory requirements result in the exclusion of microcomputing platforms as targets for software based OPC UA stacks.

978-1-5090-2339-4/16/$31.00 © 2016 European Union

To semantically integrate actuators and sensors driven by low cost, well suited microprocessors into a vertical communication hierarchy, these devices must be capable of providing a detailed, specification conformant information model of themselves. “Namespace 0”, a meta-model of the OPC UA modeling mechanism [1], [5], must consequently be equally embedded into these devices as a vendor and function specific device model. The persistent storage demands of the default XML representation of namespace 0 [8] surpasses the available EEPROM or Flash sizes in embeddable controllers by several orders of magnitude. Parsing of natural language based data formats, such as XML, are equally problematic due to the necessity to represent the parsed data in memory in order to access it efficiently.

The authors recently presented an embeddable OPC UA hardware server IP Core (with focus on the hardware architecture) [9], [10] to overcome these limitations and permit actors and sensors to be semantically embedded into information networks. To enable this technology, a dense binary encoding for OPC UA data representation was created to allow the usage of standard EEPROM as persistent data storage. The first part of this paper will introduce this derived binary format and discuss the measures used to code data processing friendly structures while maintaining high encoding density. Given both slow legacy field bus technologies and crowded networks arising from the introduction of new instrumentation in the near future, the second part of the paper examines the application of transport compression to OPC UA as means of bandwidth reduction. [11]

II. Binary namespace encoding

OPC UA represents objects, types, variables and methods by modeling a node-based graph with typed references indicating relations between said nodes. The specification details the attributes and reference semantics required by this information modeling approach. The in-memory representation of nodes and references is not specified by OPC UA , giving stack implementations the choice of employing whatever means the programming language provides to create and manage instances of nodes and their references. At runtime, the OPC UA server accesses this data model on behalf of the client or user application, making an efficient means of finding nodes and retrieving their attributes essential to OPC UA servers.

978-1-5090-2870-2/16/$31.00 ©2016 IEEE

Subsequent examination focus solely on the namespace 0 data model as defined by the OPC UA specification.[8] Results remain transferable to any application specific data model.

The non-volatile memory required to store the XML definition of namespace 0, measuring at 1,3 MB and containing 1514 nodes with 5666 references, already makes single chip or EEPROM based storage approaches unfeasible. Further more, the computing power required to parse the XML format and the memory needed to represent this data during runtime is simply not present in most microcomputing platforms. Microcomputing platforms would require a "pre-parsed" binary data representation of the information that can be mapped to common, serial 8 Bit 128-256 kB EEPROM products. The simplest way to obtain a binary representation of OPC UA data models is to employ the specifications binary transport encoding. [12] Encoding namespace 0 by said means results in a 197 kB binary image [1]. The resulting binary image has all properties of OPC UA 's transport encoding, of which the serial decoding prerequisite proves most problematic: data fields are aligned in such a manner that all data stored prior to the byte being read must have been parsed in order to derive the type and size of the following field.

The goal of the derived binary model representation (binary mapping) is to allow a process to search and read data directly from that representation, without the need to create an internal representation of the data. Using the default binary transport encoding as a template, optimizations were introduced to alleviate its shortcomings. A python based compiler was written to automatically parse and process XML descriptions of data models and convert them into the binary mapping subsequently described. The program is explicitly a compiler, not a model transformation tool, since creating an efficient binary image requires introspection of OPC UA datatypes, model semantics and address based linking. Figure 1 depicts an example of the data encoding generated for generic nodes as produced. A modified form of the compiler was contributed to the open62541 OPC UA stack as a code generator.

A. Data alignment

Numerical values in OPC UA binary are encoded in little-endian format. In addition, some fields, such as strings, are not aligned to word boundaries and have variable length. A fixed width processing architecture is forced to correct field alignments at its native word size boundary. While platforms using RAM and a MMU can easily correct for these alignement issues, architectures using local registers must careful inspect field length and data type sizes when reading OPC UA binary data. The derived binary encoding assumes that any processing architecture will process the data in byte increments. Numerical fields are always encoded in Motorolla-Byte-Order to allow clear-&-(left)shift register-chains to easily process integers of different sizes. This encoding strategy removes the requirement for alignment corrections independently of the fields length, while read numerical values are immediately usable for calculations.

B. Address width of a linear binary mapping

OPC UA nodes are identified and referenced by using the respective targets node ID. Locating a node thus requires an indexed search operation through all nodes in an image. By using a direct addressing scheme in a binary mapping, locations of nodes can be directly referenced, making lengthy search operations obsolete, effectively making a node retrievable with O(1) complexity.

In order to determine the address width to be used in binary mapping, reasonable size barriers must be established. Coarse characteristics of a low redundancy binary representation can be derived by using lossless data compression on the namespace 0 XML definition, which provides a binary image with minimal redundancy. Of the examined algorithms[2], the Burrow-Wheeler algorithm[13] reduced the memory footprint to a minimum of 79 kB. Given the target of 8-Bit linear memory and the alignment requirement, possible address width increments are 1 Byte. A low entropy binary encoding is given to produce images larger than 64 kB (2 byte addresses). The next larger address width of 24 bit was thus chosen for referencing positions in a linear memory, allowing for a maximum image size of 16 MB.

C. Attribute recoding

The default OPC UA binary transport encoding is neither efficient nor particularly machine friendly. The main consideration for optimizing parsing of the encoding concerns the representation of nodes and their attributes. Variable contents ("data values") were not changed and remain in their transport encoding to allow their direct insertion into OPC UA messages.

As primary means of reducing interpretation overhead and state machine complexity in parsing processes, all boolean node attributes were recoded as bits instead of 32bit values. The number of namespaces was reduced to 4 and only the usage of 16 bit numeric node IDs is permitted. Node ID and all binary attributes of all node types can thus be encoded in the first 3 bytes of a node.

To improve parsing times, a reading process must be able to skip data segments. To enable this feature, offsets to specific data inside a node was inserted. A 2 byte offset value indicates where the node type specific "payload" data is encoded. A similar 2 byte offset before that payload identifies the size of the data and thereby the position of the next node. Similar offsets were added to permit circumventing references and string attributes. The width of these relative offsets define the maximum data size that can be stored, reducing the maximum data size of any nodes type specific attributes to 64 kB. In addition, string sizes for browseName, qualifiedName and descriptions were reduced from 12 GB to 8 kB in total.

Each OPC UA node holds an average of 3.75 references consisting of 7 bytes. Recoding references thereby not only

[1]Excluding certificates; metric derived using the open62541 stack revision 7fb332d

[2]LZ77, LZW, Huffmann, Burrow Wheeler

severely decreases parsing complexity, but saves considerable resources. References encode their targets using the direct addresses of their target node. Since only a limited set of addresses are eligible for the reference type field, indirect addressing is used to encode type information. Each reference uses 6 bit $A_{indirect}$ address to encode its type. Memory locations after byte 2 of the binary mapping are considered to be consecutive 24 bit addresses, referred to as slots. The namespace compiler determines common type nodes throughout data models and uses these slots to encode their addresses. The direct address of a slot $A_{indirect}$ corresponds to $A_{direct} = (A_{indirect} + 1) * 3$. The first slot (address 0) always holds the first encoded nodes address. Slots are also employed when encoding DataTypes and VariableType nodes.

To allow a parsing processes to find sources of monodirectional references, the namespace compiler always creates bidirectional references, using the additional boolean attribute "hidden" to declare a reference as not being part of the data model. A server does not reveal these references to clients when browsing, but can use them to effectively navigate the data model.

D. Synchronization Markers and Node Ordering

If a process starts reading the binary image at a random position, the field type at said position is unknown and any subsequent data thereby becomes undecodable. This serial decoding prerequisite forces reading processes to decode all data of the binary image in order to locate a particular information. One mechanism that can circumvent this problem would be indexing the node addresses as they are being decoded. This solution however would require at least 3 Byte RAM - one 24 bit address - per indexed node.

As an alternative, synchronization markers are inserted after every k^{th} node. A synchronization marker consists of a CRC16 checksum of the prior j bytes. A linear reading process can thereby locate a nodes start by continuously comparing the checksum of the prior $j + 2$ read values with the last value read from the image. If the last value matches the 16 bit checksum, a node start has been located and decoding can continue. Defaults of $k = 8$ and $j = 16$ were chosen for further comparisons based on empirical evidence, however both values are adjustable when compiling binary mappings.

Given a search beginning at a random location of the binary mapping, the reading process can now locate the beginning of nodes. By ordering nodes in the mapping, the reading process can determine if the searched node is located before or after the current node. Given that all node IDs are required to be numeric, the simplest ordering mechanism is an incremental sorting. Several well known algorithms exist that can perform search operations on ordered lists, usually with complexity $O(log(n))$.

Given the use of direct addressing, traversing edges does not require searches. However software based servers (such as open62541) that do not employ direct linked references require topological sorting, which implies that every node being referenced has already been instantiated. Since OPC UA data models always contain cyclical graphs due to all nodes being typed, maintaining this premise is not possible for all nodes. Non the less, the python compiler supports topological sorting using a modification of Dijkstra's minimal dependance sorting algorithm [14] for cyclical graphs for such use cases.

E. String attribute compression

Of the 197 kB of data in namespace 0, approximately 72 kB are strings contained in the browseName, displayname and description attributes of nodes. In contrast to other node contents, which need to remain easily parseable, these string can be compressed to reduce the images memory footprint. Several algorithms were considered for lossless compression, including Lempel-Ziv (LZ77), Lempel-Ziv-Welch (LZW) and Burrow Wheeler. Though all three algorithms show remarkable compression ratios for large natural language strings, they proved to be unsuitable for encoding distributed strings with less than 100 symbols. This is due to the fact that all the given algorithms operate on a block based scheme with distinct dictionaries, a technique becoming ineffective when the data being encoded is smaller than the algorithms block length. In addition, decompression algorithms proved to be too time consuming or resource intensive for 8-Bit platforms and hardware decoders.

For compressing strings in attribute fields, a standard Huffmann encoding was chosen. The benefit of this encoding is the global, detachable decoding table that can be prepended to the generated binary image. Since the algorithm is not block-based, any string can be losslessly compressed at any point in the image. Construction of the encoding table is handled by the namespace compiler and incorporates statistics of all string attributes in the servers information space. The string length field was modified to contain 3 bits indicating the number of padding bits added and a 13 bits indicating the number of consecutive bytes encoding the compressed string. Applying the Huffmann-encoding to string attributes of nodes reduced the binary image size from 147 kB to 116 kB while maintaining efficient decodability.

III. Transport compression

While storing model information in a fashion beneficial to hardware can easily be solved by changing the information encoding to better suite linear memory operations, maintaining a standard conformant transport encoding is mandatory for interoperability. The relatively inefficient OPC UA binary encoding can represent a problem when (i) the network technology offers only very limited bandwidth or (ii) the network is crowded by communications reducing the available bandwidth. In regard to the increasing use of "smart" field devices in industrial networks and the onset of semantic integration of the field device layer, it can be expected that bandwidth will remain a critical resource.

To evaluate the problem, a server/client pair was written that performed a series of typical steps in OPC UA interaction. After establishing a connection, the client would

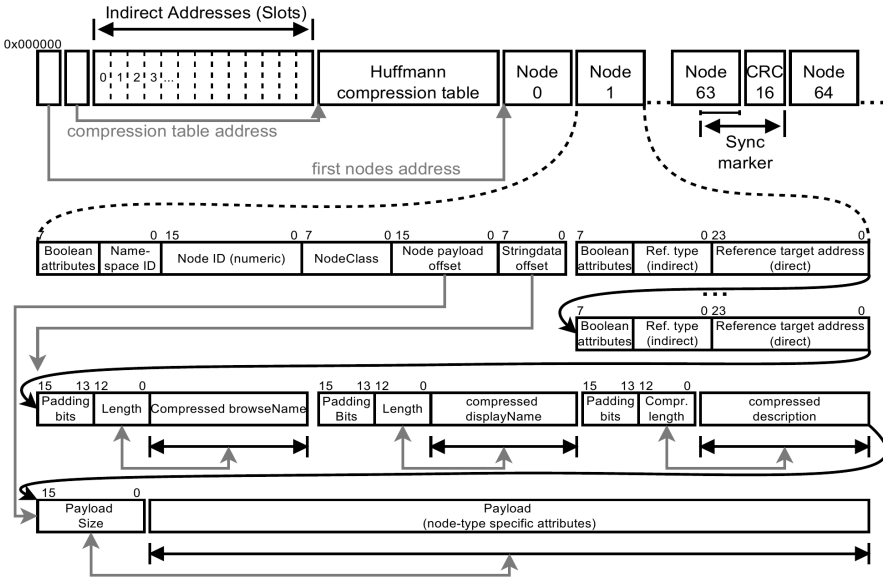


Figure 1: Simplified structure of the encoded binary image using string compression (top) and encoding details for standard node attributes (bottom).

1) Browse five nodes in the RootObjects folder of the server, reading all attributes of those nodes.
2) Register a subscription and a single monitored item for a variable node in the RootObjectes folder.
3) Read and write a 32 Bit integer from/to a node. This operations causes a single changeNotification for the previous subscription.
4) Invoke a method node, passing a 11 symbols long string argument and obtaining one 32 bit integer as method result from the server.

These four steps represent routine server/client a typical communication scenario with OPC UA enabled devices. The data communicated in this testcase serves as a reference point for the subsequent considerations[3] and amount to 7.4 kB bulk traffic in 19ms over a 100 $\frac{MBit}{s}$ Ethernet connection (389 $\frac{kB}{s}$). Measurements (see figures 2 and 3) based on bulk data consider this traffic as a single sequence, while packet based measurements compress packets individually.

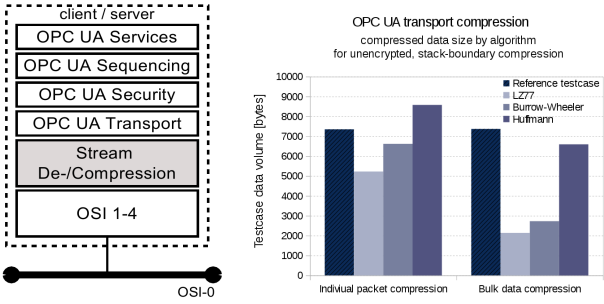
Given that the memory footprint of binary namespaces could be significantly reduced, it was attempted to also reduce the communications bandwidth. To that end, it was theorized that a compression mechanism could be positioned in the protocol stack to compress the transport data. [11] In regard to systems with limited memory resources, only compression algorithms acting on a single packet were qualified as possible implementations. In particular, media specific delta-algorithms that analyze and exploit patterns in specific repetitive communication schemes, such as read or write requests of the same node, were not considered. Four mechanisms for compression (detailed in the following sections) using the bzip2, gzip and huffman libraries, implementing the Burrow-Wheeler, LZ77 and Huffmann compression algorithms respectively, were analyzed.

[3]Ethernet, IPv4 and TCP headers are not included in the traffic measurement

A. Stack-boundary compression

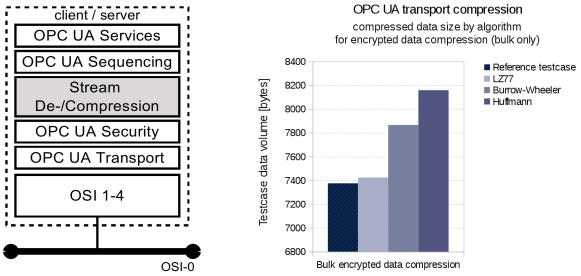
OPC UA stacks such as open62541 offer access to message handling interfaces between the OPC UA protocol layers and the underlying OSI-4 layer. The undoubtedly simplest form of compressing OPC UA communications can be implemented by having this interface compress data outbound from the stack and decompress data inbound to the stack. In order to enable the usage of generic server/clients in combination with stream compressing server/clients, a mechanism to recognize the presence and type of compression at a transport message level is required. This was accomplished by prefixing compressed data with a string, which can be treated as an extension of the specification defined HEL, OPN, MSG or ACK prefixes of the OPC UA transport level. The prefixes “OPCBZ”, “OPCGZ” and “OPCHF” were chosen to indicate the algorithm used. In absence of a prefix, a standard OPC UA message is assumed and processed.

Compressing the transport stream in this manner did not yield significant results, as the bulk of OPC UA messages had a size between 100-200 Bytes. The size of the block compression table included in compressed messages therefore negates the benefit of the size reduction gained from compressing the transport stream. In isolated cases the size of the packets transferred over the network even became larger than their uncompressed counterparts. The size of large packages containing server or client certificates was however reduced, so that the net communication bandwidth used by the described testcase was slightly reduced (see figure 2). To demonstrate the effect small packages had on the compression algorithms, the bulk of the testcase data was captured and subjected to the



(a) Position in stack (b) Compression results

Figure 2: Stack-boundary compression.



(a) Position in stack (b) Bulk compression results

Figure 3: Pre-Encryption compression.

same compression schemes as one file - yielding expectable compression ratios of ca. 1:3.

When encoding strings in the namespace data of OPC UA , the Huffmann algorithm was able to yield good compression results for string attributes, as the huffman compression dictionary could be computed once and included a single time at the beginning of the namespace. In case of transport compression however, that compression dictionary must be included in each message, negating this approach.

B. Compression gateways

The previous two approaches operated under the assumption that the programmer has access to both the server and client stacks in order to implement the compression mechanism. This may not be the case for third party stacks embedded in proprietory devices. An approach tested for such devices was compressing gateways for unencrypted OPC UA communications. A gateway would offer a standard tcp/ip service to OPC UA clients and accept compressed messages, decompress them and subsequently forward the message to the device server. As the server would reply to the gateway, the reverse procedure would be followed. In this manner, a compressing client would be able to use tcp/ip compression gateways to communicate with legacy or third party devices. The scenario can be reversed (compressing server, non-compressing client) if desired. In terms of bandwidth reduction, tests and yielded identical results to those of direct compressed communications, as the volume and data contents of the compressed packets remained unchanged. To gain any benefit from this approach, compression gateways must span different subnets, as otherwise the data is transmitted multiple times over the same network and any compression benefit is negated.

C. Encrypted transport compression

Encrypted OPC UA communications differ from the hereto examined messages by including a signature and applying a cypher algorithm to the message body. Inclusion of a signature leads to an increase of data of several hundred bytes per message, which would in principle counteract the previously described problems when compressing small messages. However, modern encryption algorithms such as AES are designed not to generate predictable or repetitive data in order to hinder cryptanalysis. It is exactly this property that has a large impact on compression algorithms, which would find few (if any) redundant data in an encrypted message. As an example, the bulk testcase data generated by the testcase was compressed reducing the message size from 7.3 kB to 2.7 kB[4]. When that bulk testdata is encrypted using aes-256-cbc and then compressed, the resulting data has a size of 7.8 kB, showing that the pseudo-randomized structure the file is simply unsuitable for block based compression algorithms (see figure 3).

The only form to remedy this problem would be to compress the data prior to its encryption. As this message part will not include the UA transport, security or sequencing headers, the message being compressed will be even shorter than those tested in the previously unencrypted communications scenario. Though this was not explicitly tested, it can be expected that compressing data prior to its encryption will yield worse than that in unencrypted communications.

D. Compression as a service

A solution considered was to selectively compress large, structured data reads when the application expects that the data may be well compressible. This procedure would delegate the choice of when and how to compress the data to the userspace application, while the OPC UA server would implement a compression facet. A client message to this compression service would consist of a serialized format of the actual service request (such as “attribute/read”) in combination with a desired compression algorithm and its parameters. The server would then execute the actual service and compress the response, which would then be encapsulated within a compression service response and passed back to the client.

Compression as a service (see figure 4) offers the additional benefit that information pertaining to the algorithm used can be included in the service response. This would allow a server to evaluate whether compression yields beneficial results and choose a compression mechanism - including no compression at all - that best suites the data. The client would in this case only specify which algorithms are supported, not which are to be used.

[4]bulk data, compressed using bzip2

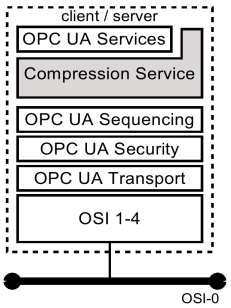


Figure 4: Compression service as a server facet.

The usage of compression as a service breaks with the hardware model derived for the limited-resources scenario and was therefore not further analyzed. It however remains a possibility for software stacks in general.

IV. Conclusion

This paper analyzed encoding schemes for OPC UA data with a special focus of making OPC UA accessible to devices with severely limited memory and computing resources. Examination of the namespace storage facilities yielded an alternate binary information model encoding suitable for small, 8 bit linear EEPROM or Flash memory products. The described encoding scheme and string compression were able to reduce the memory requirements from 197 kB using the OPC UA binary encoding scheme to 116 kB, requiring only 56% of the otherwise necessary non-volatile memory. Additional information such as synchronization markers, direct and indirect addressing and relative offsets enable retrieving information with considerable ease.The encoding can be automatically generated from XML descriptions using a Python based compiler. The compiler, being in principle a model analyzer, was also contributed to the open62541 project to generate namespace related C code.

To lower the traffic generated by OPC UA connections, single-message algorithms for transport compression were closer examined. Stack boundary compression of unencrypted data, in-stack compression of encrypted data and the usage of compression gateways all yielded suboptimal results for this protocol. This was mainly attributed to relatively small messages with a highly redundant structure where the size of the compression dictionaries outweighed the actually compressed data. A compression-as-a-service approach, which could remedy this problem, was introduced but not analyzed in detail, as it did not befit the device-class targeted by the optimization process.

Future works will continue to focus on the efficient encoding of OPC UA namespace data, in particular relating to the implementation of the OPC UA-Realtime logic of the OPC UA peripheral IP. One mayor shortcoming currently being addressed is the problem of adding data to the static namespace image, for example for adding new nodes or references. To that end, filesystem-like block (re-)allocation mechanisms are being examined. Transport compression is, as a result of this papers findings, not being further pursued.

References

[1] M. Damm, W. Mahnke, and S.-H. Leitner, *OPC Unified Architecture*. Springer, 4 2009.

[2] L. Zheng and H. Nakagawa, "Opc (ole for process control) specification and its developments," in *SICE 2002. Proceedings of the 41st SICE Annual Conference*, vol. 2, p. 917–920 vol.2, 8 2002.

[3] S. H. Jeanne Schweder, "Platform industrie 4.0 proposes reference architecture model for industrie 4.0: Opc-ua confirmed as one and only standard in category "communication layer"," 04 2015.

[4] M. Graube, J. Pfeffer, J. Ziegler, and L. Urbas, "Linked data as integrating technology for industrial data," in *Network-Based Information Systems (NBiS), 2011 14th International Conference on*, p. 162–167, Sept 2011.

[5] L. Urbas, *Process Control Systems Engineering*. Deutscher Industrieverlag, 2012.

[6] G. Shrestha, J. Imtiaz, and J. Jasperneite, "An optimized opc ua transport profile to bringing bluetooth low energy device into ip networks," in *Emerging Technologies Factory Automation (ETFA)*, 9 2013.

[7] J. Imtiaz and J. Jasperneite, "Scalability of opc-ua down to the chip level enablesinternet of things," in *Industrial Informatics (INDIN)*, 7 2013.

[8] OPC Foundation UA Working Group, "Opc foundation namespace 0 xml definition." https://opcfoundation.org/UA/schemas/1.02/Opc.Ua.NodeSet2.xml.

[9] C. P. Iatrou, S. Hoeppner, M. Graube, and L. Urbas, "Entwurf und implementation eines opc ua hardware stacks," in *Kommunikation in der Automation (KommA)*, 2016.

[10] C. P. Iatrou and L. Urbas, "Opc ua hardware offloading engine as dedicated peripheral ip core," in *World Conference on Factory Communication Systems (WFCS)*, May 2016.

[11] L. C. Kho, S. S. Ngu, Y. Tan, and A. O. Lim, *Application of Data Compression Technique in Congested Networks*. Springer, 2016.

[12] VDK/VDE, *OPC-UA Specification Part 6*, vol. 6 of *OPC-UA Specification*. OPC Foundation, 11 2008.

[13] U. Lauther and T. Lukovszki, "Space efficient algorithms for the burrows-wheeler backtransformation," in *Algorithms ESA 2005*, Lecture Notes in Computer Science, Springer-Verlag, 2005.

[14] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, p. 269—271, 1959.